qual_by_position.py 사용 설명서

FASTQ 위치별 평균 품질 점수 계산 · lab11/src/qual_by_position.py

1. 무엇을 하는 프로그램인가

FASTQ 파일의 모든 read 에서 같은 위치(1번째 염기, 2번째 염기, …)의 품질 점수를
모아 위치별 평균을 계산하고, '위치<TAB>평균품질' 형식으로 화면에 출력한다.
FastQC 의 'Per base sequence quality' 그래프와 같은 값을 직접 계산해 보는 실습이다.

2. 실행 방법

src 폴더로 이동한 뒤 FASTQ 파일 경로를 인자로 주어 실행한다.

```
cd $MYHOME/lab11/src
python3 qual_by_position.py ../data/SRR1553607_1.fastq
```

결과를 파일로 저장하거나 일부만 보고 싶을 때:

```
# 결과를 TSV 파일로 저장
python3 qual_by_position.py ../data/SRR1553607_1.fastq > ../doc/qual_by_position.tsv

# 앞 10개 위치만 보기
python3 qual_by_position.py ../data/SRR1553607_1.fastq | head

# 평균 품질이 30 미만인 위치만 골라 보기
python3 qual_by_position.py ../data/SRR1553607_1.fastq | awk -F'\t' '$2 < 30'

# 트리밍 전후 비교 (위치, 원본, 트리밍 후)
paste <(python3 qual_by_position.py ../data/SRR1553607_1.fastq) \
      <(python3 qual_by_position.py ../data/trimmed.fastq | cut -f2) | head
```

인자를 주지 않거나 두 개 이상 주면 사용법을 출력하고 종료한다.

```
$ python3 qual_by_position.py
사용법: python3 qual_by_position.py <FASTQ 파일>
```

3. 입력 형식

한 read 가 4줄로 이루어진 일반 FASTQ 파일 (압축되지 않은 .fastq).
품질 문자는 Phred+33 인코딩이라고 가정한다 (Illumina 1.8 이상, SRA 데이터).

```
@SRR1553607.1 1 length=101          <- 1줄: 이름
GTTAGCGTTGTTGATCGCGACGCAACAAC...    <- 2줄: 서열
+SRR1553607.1 1 length=101          <- 3줄: 구분자
BB@FFFFFHHHHHJJJJJJJJJJJJJJJ...    <- 4줄: 품질 (문자 1개 = 염기 1개)
```

4. 출력 형식

두 열이 탭으로 구분되어 있다. 1열은 위치(1부터 시작), 2열은 평균 품질(소수 둘째 자리).

```
위치 평균품질   (헤더는 출력되지 않음)
1 32.04
2 32.17
3 32.28
4 35.85
5 35.65
...
99 29.03
100 28.93
101 28.25
```

위 예시는 ../data/SRR1553607_1.fastq 를 실행한 실제 결과이다.

5. 계산 방법

- 품질 점수 = ord(품질 문자) - 33 예) 'J' → 74 - 33 = 41, 'B' → 66 - 33 = 33
- sums[i] 에 i번째 위치 점수를 더하고, counts[i] 에 그 위치에 염기가 있는 read 수를 센다.
- 위치 i 의 평균 = sums[i] / counts[i]
- read 길이가 서로 달라도 된다 (트리밍된 파일). 위치마다 실제 염기가 있는
- read 수로만 나누므로, 짧은 read 때문에 뒤쪽 평균이 낮아지지 않는다.

간단한 예 — read 3개:

```
read1 품질: I I 5      -> 40 40 20
read2 품질: I 5 5      -> 40 20 20
read3 품질: 5 5        -> 20 20       (길이 2)

위치1 = (40+40+20)/3 = 33.33
위치2 = (40+20+20)/3 = 26.67
위치3 = (20+20)/2    = 20.00        <- read3 은 제외
```

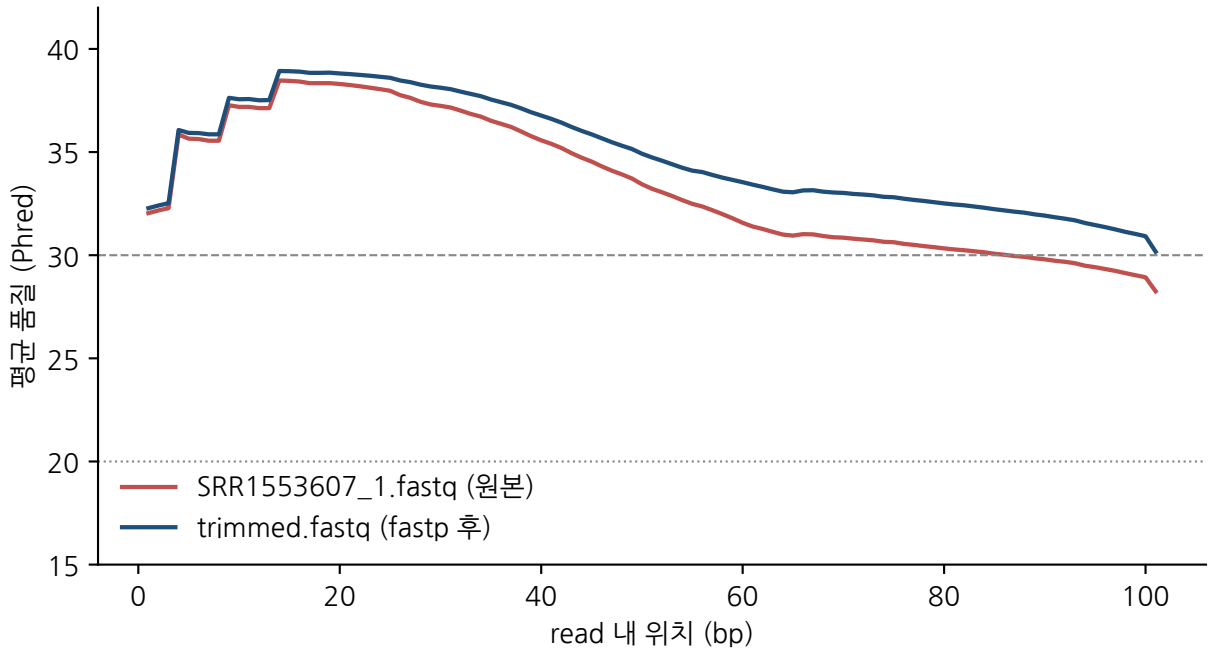
6. 함수 구성

- read_fastq(filename) : 4줄씩 끊어 (이름, 서열, 품질) 튜플 리스트로 돌려준다 (qual_summary.py 와 같은 방식).
- quality_by_position(reads) : 위치별 평균 품질 리스트를 돌려준다 (0번 = 1번째 위치).
- 다른 스크립트에서 import 해서 쓸 수 있다:

```
from qual_by_position import read_fastq, quality_by_position
means = quality_by_position(read_fastq('../data/trimmed.fastq'))
```

7. 실행 결과 예시 — 트리밍 전후 비교

lab11 데이터로 계산한 결과를 그래프로 그리면 다음과 같다 (그래프는 설명용).



- 원본은 처음 몇 염기를 지나 약 36~38 로 높다가, 뒤로 갈수록 떨어져 101번째에서 약 28 이다.
- 트리밍 후에는 끝부분 품질이 약 30 으로 올라간다 — 낮은 품질 끝부분이 잘려 나갔기 때문.
- 보통 Q30(오류율 0.1%) 이상이면 좋음, Q20(오류율 1%) 미만이면 트리밍을 고려한다.

8. 주의할 점

- 파일 전체를 메모리에 읽는다 (readlines). 수백 MB 이상의 큰 파일은 오래 걸릴 수 있다.
- .fastq.gz 압축 파일은 바로 읽지 못한다 → gunzip 으로 먼저 풀어서 사용.
- Phred+64 (옛 Illumina) 파일은 점수가 31 높게 나온다.
- 외부 라이브러리 없이 표준 라이브러리(sys)만 사용한다.